Live Presentation Script & Teacher Demo Guide

Mathematical Foundations of 2D to 3D View Synthesis: Utilizing Linear Algebra & Projective Geometry

Team Members: Shreyansh Raj • Shaurya Singh • Sarthak Bhowmik

Audience: Mathematics Course Instructor • **Duration:** 5–7 Minutes

> **CRITICAL PRESENTATION STRATEGY:**
> Lead with **Pure NumPy Linear Algebra & Projective Geometry**, NOT deep learning or PyTorch. Clarify that PyTorch is strictly isolated to a single 1-step depth prior. **100% of our linear algebra engine was written from scratch in pure NumPy!** All 3D spatial unprojections, SO(3) Lie group rotation kinematics, focal pivot translations, planar homographies H_k, and back-to-front alpha compositing were mathematically formulated and programmed by our team.

Part 1: Pre-Demo Setup & Checklist

- **Launch Backend & Frontend:** Open terminal in project root and run:
  `uvicorn src.api.server:app --app-dir v1 --port 8000`
- **Browser Navigation:** Open Google Chrome / Edge to `http://localhost:8000` and press **F11** (Fullscreen).
- **Initial State:** Have **Sample 4 (Espresso Cup)** or **Sample 1 (Living Room)** already loaded so parallax is responsive immediately.
- **Inspector Drawer:** Keep the **Math Inspector** drawer toggle accessible to display live matrices instantly.

Part 2: Verbal Script & Action Walkthrough

Act 1: The Hook & Problem Statement (0:00 – 1:00)

[SCREEN: MAIN APPLICATION VIEWPORT WITH INTERACTIVE CANVAS CENTERED]

> *"Good day, Professor.*

Today our team—Shreyansh Raj, Shaurya Singh, and Sarthak Bhowmik—is demonstrating our project:

> ***Mathematical Foundations of 2D to 3D View Synthesis**.*

> *In traditional 3D computer vision, view synthesis requires either multi-camera LiDAR setups or generative neural*

networks that hallucinate pixels unpredictably. Our goal was different: ***we built a deterministic mathematical engine that formulates 3D spatial geometry strictly through Applied Linear Algebra and Projective Geometry***.

Given a single 2D RGB image matrix, our engine models the unprojection of the projective plane P^2 into a 3D Euclidean viewing frustum R^3, calculates rigid-body kinematics in the 3D rotation group SO(3), derives planar homographies H_k, and synthesizes real-time parallax views with continuous mouse interaction."

Act 2: The Linear Algebra Pipeline (1:00 – 2:45)

[SCREEN: OPEN THE MATH MATRIX INSPECTOR ON THE RIGHT PANEL]

"Our pipeline is divided into three interconnected mathematical stages, corresponding to our delegated roles:

1. Disparity Mapping & Depth Field Analytics (Led by Shreyansh):

An image is initially an H × W × 3 tensor. Shreyansh's module processes a continuous scalar disparity field d(u, v) and converts it to metric Euclidean depth via the inverse relation Z(u, v) = 1 / d(u, v). To prevent color bleeding at depth edges, he implemented an edge-preserving guided filter that computes local covariance using O(1) box-filter integral images, aligning depth discontinuities with image gradients and inpainting occlusion voids.

2. Projective Geometry & Camera Transformations (Led by Shaurya):

Shaurya parameterized the pinhole camera intrinsic calibration matrix K and derived its analytical inverse K^-1. He designed our vectorized unprojection engine, solving:

$$[X, Y, Z]^T = Z(u, v) \cdot K^{-1} \cdot [u, v, 1]^T$$

simultaneously across all pixels via NumPy tensor broadcasting. When the virtual camera orbits, Shaurya models the rotation in the Special Orthogonal group SO(3) via composite Euler rotations R = R_y(θ_y) R_x(θ_p), continuously verifying that R R^T = I and det(R) = +1. Furthermore, he derived a focal pivot translation vector t = (I - R)c that anchors camera motion around the subject rather than tumbling around the origin.

3. Multi-Plane Image Homographies & Alpha Compositing (Led by Sarthak):

Finally, Sarthak handled the view synthesis using Multi-Plane Images (MPI). Rather than using naive point projection, he slices the 3D volume into L discrete planes spaced uniformly in disparity space. For every plane k at distance d_k, Sarthak derived the induced planar homography:

$$H_k = K (R - (t\, n^T) / d_k) K^{-1}$$

where n = [0, 0, 1]^T. He inverts H_k for backward coordinate resampling, and composites the warped planes

> *back-to-front using Porter-Duff alpha algebra:*

$$C_{out} = C_k + (1 - \alpha_k) C_{acc}$$

> *This cleanly resolves occlusions and transparency without depth cracking."*

Act 3: Live Interactive Demonstration (2:45 – 4:45)

[ACTION 1: MOUSE TRACKING PARALLAX]

> *"Notice on screen: as I move my cursor horizontally and vertically, foreground subjects shift with higher displacement than background elements. This is genuine perspective parallax rendered dynamically at 60 to 120 FPS via an HTML5 canvas viewport with spring-damper cursor smoothing."*

[ACTION 2: IDLE AUTO-WIGGLE]

> *"When I release the mouse, the engine smoothly transitions to an automatic Lissajous figure-8 orbit, continuously evaluating the transformation equations to provide natural depth perception."*

[ACTION 3: LIVE MATH MATRIX INSPECTOR]

> *"Looking at the Math Inspector panel, you can see our linear algebra live in the browser. As the camera angle shifts, the rotation matrix R updates dynamically, while KaTeX verifies on-the-fly that R R^T = I_3 and det(R) = 1.000. You can also inspect the calculated 3 × 3 homography matrix H_k for any individual plane slice."*

[ACTION 4: LAYER SLICES & PRESETS]

> *"In the Layer Slices drawer, you can view the decomposed MPI planes: foreground layers isolate the front subject, while deeper layers feature inpainted background continuity. Our unified backend serves this client directly while processing new images through asynchronous Celery task queues."*

Act 4: Closing & Hand-Off (4:45 – 5:15)

> *"In conclusion, we have transformed a single 2D image into an interactive, mathematically consistent 3D viewpoint engine through rigorous applied linear algebra: K^-1 unprojection, SO(3) rotation kinematics, planar homographies H_k, and alpha algebra.*

> *Thank you, Professor. We welcome your questions."*

Part 3: Teacher Q&A Defense Guide

Likely Teacher Question	Precise Mathematical Answer to Deliver
Q1: Why use planar homographies H_k instead of direct 3D point splatting?	"Point splatting forward-projects discrete coordinates onto a novel canvas. Because projected coordinates are continuous reals, rounding creates 'cracking' (empty unmapped holes) requiring expensive heuristic buffers. A planar homography H_k is a projective bijection between planes. Using the inverse homography H_k^-1, we backward-map destination integer pixels to continuous source coordinates with bilinear interpolation. This is strictly hole-free, differentiable, and parallelizable via SIMD/GPU."
Q2: Why space MPI layers linearly in disparity (1/Z) rather than Euclidean depth (Z)?	"Under perspective projection, pixel shift Δu is inversely proportional to depth: Δu $\propto$ 1/Z. Spacing planes linearly in depth Z causes massive discontinuous pixel jumps in the foreground and redundant sub-pixel steps in the background. Uniform disparity spacing (Δd = const) ensures that the pixel parallax displacement between adjacent planes is strictly constant across the entire viewing frustum."
Q3: How do you verify the rotation matrix preserves metric shape?	"We construct R in SO(3) as the product of Givens planar rotations R_y(θ_y) R_x(θ_p). Because each elementary rotation is orthonormal with determinant +1, their product is guaranteed to satisfy R R^T = I_3 and det(R) = +1. This proves lengths, angles, and chirality are strictly preserved without shear or scaling. Our automated test suite verifies || R R^T - I_3 ||_F < 10^-6."
Q4: Did you use PyTorch for the math, or NumPy? Where does AI stop?	"We used **NumPy**, not PyTorch, for 100% of our mathematical engine! PyTorch was strictly confined to an isolated initial step to run a pretrained model to estimate a raw disparity map, which is immediately cast to a `numpy.ndarray`. Every single mathematical operation—K, K^-1, unprojection rays, SO(3) rotations, pivot vector t, homographies H_k, and alpha compositing—is implemented in pure NumPy using matrix multiplication (@) and vector broadcasting. Everything from the intrinsic calibration K, unprojection rays, SO(3) kinematics, focal translation t, homography derivations H_k, and alpha compositing was derived and implemented by our team."

Part 4: Presenter Cheat Sheet

Time	Presenter	Screen Action	Key Linear Algebra Concept
0:00–1:00	Team / Lead	Fullscreen live UI	Projective Plane P^2 → R^3 Frustum Modeling

Time	Presenter	Screen Action	Key Linear Algebra Concept
1:00–1:35	Shreyansh Raj	Depth Map toggle	Scalar field Z = 1/d, Guided filter local covariance
1:35–2:10	Shaurya Singh	Math Inspector tab	Intrinsic K, Inverse K^-1, SO(3) Orthogonality, Pivot t
2:10–2:45	Sarthak Bhowmik	Layer Slices tab	Disparity MPI Slicing, Homography H_k, Porter-Duff Over
2:45–4:45	Team	Mouse interaction & Presets	60 FPS Canvas Parallax, Bilinear Lerp, Live KaTeX Matrices
4:45+	All Members	Q&A Defense	Orthogonality checks, Inverse Homography vs Splatting